

Lecture 1: Big Data Frameworks & Apache Spark

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A

amitkdhar@iitbhlai

January 7, 2026

Big Data Framework Overview

What is a Big Data Framework?

- **Big Data Framework:** A set of tools and architectures designed to handle the **3 Vs** (Volume, Velocity, Variety).
- It moves beyond single-server capabilities to **distributed computing**.
- **Core Goal:** To ingest, store, process, and analyze massive datasets efficiently and fault-tolerantly.

The 5 Layers of Big Data Architecture

A typical modern Big Data architecture consists of logical layers:

1. **Ingestion Layer:** Getting data in.
2. **Storage Layer:** Keeping data safe and accessible.
3. **Processing Layer:** Transforming and computing.
4. **Analysis & Query Layer:** Making sense of the data.
5. **Visualization/Serving Layer:** Delivering value to users.

Parts of the Framework

1. Ingestion Layer

Description:

- The entry point for data.
- Handles data from various sources: IoT sensors, logs, databases, social media.
- Must handle **bursts** of data (Velocity).
- Decouples producers from consumers.

Software Examples:

- **Apache Kafka:** Distributed event streaming platform.
- **Apache Flume:** Aggregating log data.
- **Amazon Kinesis:** Cloud-managed streaming.

2. Storage Layer

Description:

- The "Data Lake". Stores raw data indefinitely.
- Must be scalable (Volume) and fault-tolerant.
- Supports structured and unstructured data (Variety).

Software Examples:

- **HDFS (Hadoop Distributed File System):** The classic on-premise standard.
- **Amazon S3 / Azure Blob Storage:** Cloud object stores.
- **NoSQL Databases:** Apache Cassandra, MongoDB, HBase.

3. Processing Layer

Description:

- The "Engine Room". Converts raw data into insights.
- Can be **Batch** (historical data) or **Stream** (real-time data).
- Performs cleaning, aggregation, and transformation (ETL).

Software Examples:

- **Apache Hadoop MapReduce:** Disk-based batch processing (Legacy).
- **Apache Spark:** In-memory batch and stream processing (Modern Standard).
- **Apache Flink:** True real-time stream processing.

4. Analysis & Query Layer

Description:

- Provides interfaces for humans or applications to query processed data.
- Often uses SQL-like syntax for accessibility.
- Optimizes queries for low latency.

Software Examples:

- **Apache Hive:** SQL Data Warehouse on Hadoop.
- **Presto / Trino:** Fast distributed SQL query engine.
- **Apache Drill:** Schema-free SQL query engine.

5. Visualization Layer

Description:

- The "Face" of the data.
- Dashboards, reports, and interactive applications.
- Enables decision-making based on analytics.

Software Examples:

- **Tableau / PowerBI:** Business Intelligence tools.
- **Grafana / Kibana:** Dashboarding for logs and metrics.
- **Custom Web Apps:** D3.js, React apps.

Example: Word Count

The "Hello World" of Big Data: Word Count

Problem Statement:

- Given a text corpus of Terabytes in size.
- Count the frequency of every unique word.

Challenge:

- File is too big for one machine's RAM.
- Needs parallel processing across a cluster.

MapReduce Logic

The problem is solved in two main phases:

1. Map Phase:

- Input is split into chunks.
- Each worker reads a chunk and emits (word, 1) for every word found.

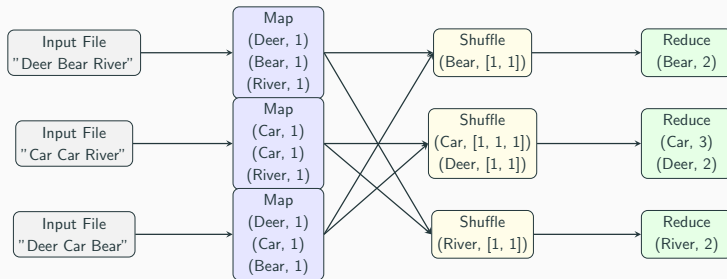
2. Shuffle & Sort:

- The framework groups all values for the same key.
- Result: (word, [1, 1, 1, ...])

3. Reduce Phase:

- Worker sums the list of 1s.
- Output: (word, total_count)

Word Count Illustration



Word Count: Python / Spark Code

```
1 from pyspark.sql import SparkSession
2
3 # 1. Initialize Spark Session
4 spark = SparkSession.builder.appName("WordCount").getOrCreate()
5 sc = spark.sparkContext
6
7 # 2. Read input file
8 text_file = sc.textFile("hdfs://.../large_corpus.txt")
9
10 # 3. FlatMap: Split lines into words
11 words = text_file.flatMap(lambda line: line.split(" "))
12
13 # 4. Map: Create (word, 1) pairs
14 word_pairs = words.map(lambda word: (word, 1))
15
16 # 5. ReduceByKey: Sum values for same keys
17 word_counts = word_pairs.reduceByKey(lambda a, b: a + b)
18
19 # 6. Action: Collect results
20 results = word_counts.collect()
```

Apache Spark

Introduction to Apache Spark

- A unified analytics engine for large-scale data processing.
- Developed at UC Berkeley AMPLab in 2009.
- Open Sourced in 2010, Top-level Apache project in 2014.
- Designed to be fast for **iterative algorithms** and **interactive queries**.



Benefits of Apache Spark

1. **Speed:** Up to 100x faster than Hadoop MapReduce in memory, 10x on disk.
2. **Ease of Use:** APIs in Java, Scala, Python, R, and SQL.
3. **Generality:** Combine SQL, streaming, and complex analytics (ML) in the same application.
4. **Runs Everywhere:** Hadoop, Apache Mesos, Kubernetes, Standalone, or in the Cloud.

Why is Spark Faster?

MapReduce (Disk Based)

- Reads from Disk.
- Maps.
- **Writes to Disk** (Shuffle).
- Reads from Disk.
- Reduces.
- **Writes to Disk.**

Spark (In-Memory)

- Reads from Disk/RAM.
- **Keeps intermediate data in RAM.**
- Chains multiple operations.
- Writes to Disk only when done (or if RAM spills).

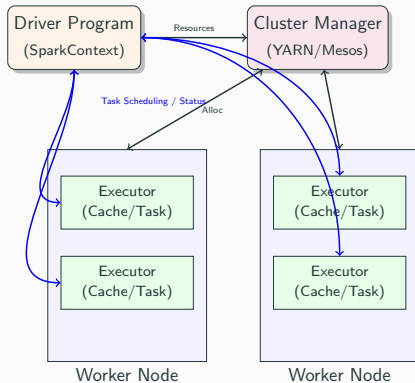
Spark Architecture

Spark Run-time Architecture

Spark uses a **Master-Slave** architecture.

- **Driver:** The process that runs the 'main()' function of the application and creates the 'SparkContext'.
- **Cluster Manager:** Allocates resources across the cluster (e.g., Standalone, YARN, Mesos).
- **Executors:** Processes running on worker nodes that execute the tasks and store data.

Spark Architecture Illustration



The Driver

- The "Brain" of the operation.
- Converts the user program into units of physical execution called **Tasks**.
- Schedules tasks on executors.
- Maintains all metadata about the application.
- **Important:** The Driver should be close to the Worker nodes (network-wise) to reduce latency.

The Executors

- The "Muscles" of the operation.
- Run the tasks assigned by the Driver.
- Return results to the Driver.
- Provide in-memory storage for RDDs/DataFrames (Caching) that are reused across applications.
- **Isolation:** Each application gets its own set of executor processes.

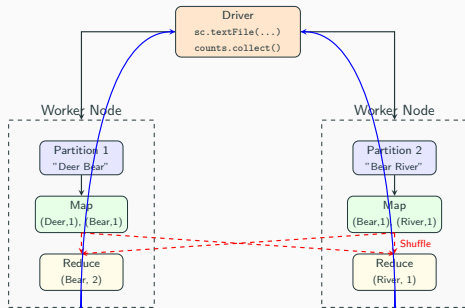
Spark Word Count Execution

Recap: Word Count Code

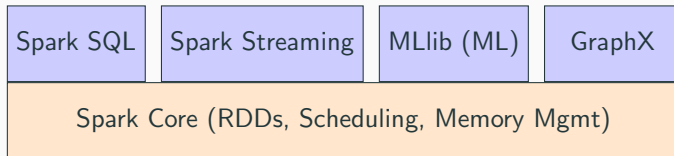
```
1 from pyspark.sql import SparkSession
2
3 # Driver Program Initialization
4 spark = SparkSession.builder.getOrCreate()
5 sc = spark.sparkContext
6
7 # Transformation (Lazy)
8 counts = sc.textFile("hdfs://.../large_corpus.txt") \
9     .flatMap(lambda line: line.split(" ")) \
10    .map(lambda word: (word, 1)) \
11    .reduceByKey(lambda a, b: a + b)
12
13 # Action (Triggers Execution)
14 results = counts.collect()
15
```

How does this code run on the Cluster?

Execution Flow on Cluster



Spark is not just a MapReduce replacement; it's a library.



Questions?